

Sistemas Operacionais

Introdução

O que é um Sistema Operacional?

- Um software que abstrai as complexidades do hardware de um usuário/programador
- SO visto como uma **Máquina Virtual** (Extendida)
 - Detalhes de hardware tais como velocidade de gravação e posicionamento de cabeça de gravação de um HD tornam-se completamente transparente ao usuário
- SO visto como um **Gerenciador de Recursos**
 - Provê acesso ordenado aos elementos de hardware
 - Monitora quem está usando qual recurso a fim de aceitar solicitações de recursos, contabilizar estatísticas de uso, e mediar solicitações conflitantes de programadores e usuários distintos

Sistema de Computador em Camadas?

- O SO representa a interface entre programas aplicativos bem como utilitários e o hardware
- Os utilitários normalmente vêm com o SO para facilitar o desenvolvimento de programas
- Tarefas como gerenciamento de arquivos, controle de I/O e criação de aplicativos fazem parte dos utilitários

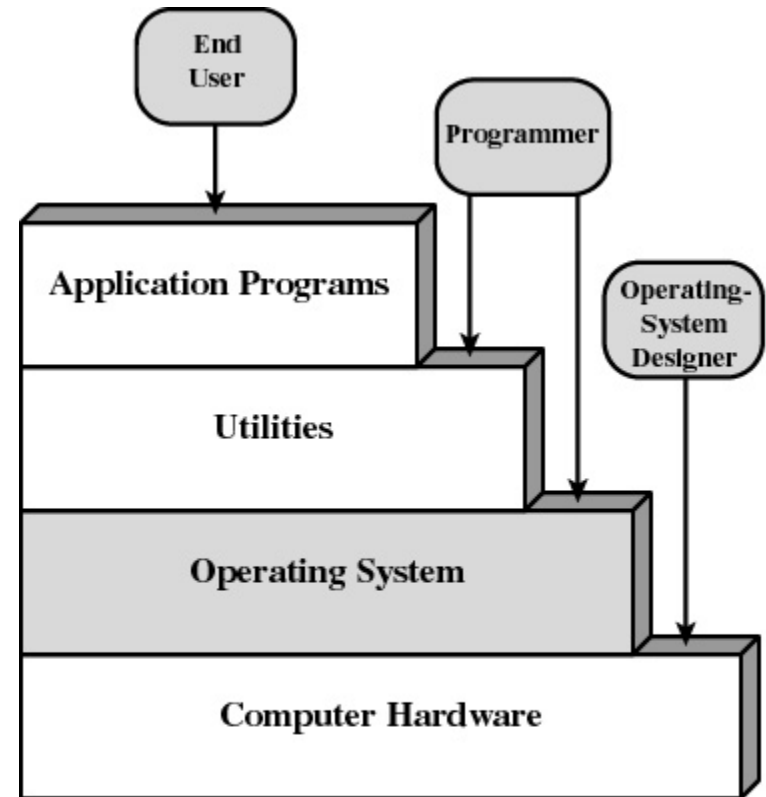


Figure 2.1 Layers and Views of a Computer System

Evolução dos Sistemas Operacionais

■ Anos 40

- Programador é também operador
- Programação é feita inteiramente em linguagem de máquina por intermédio de painéis de ligação (*plugboards*)
- Painéis de interligação evoluem para cartões perfurados
- Um programa (job) em execução ocupa toda a máquina

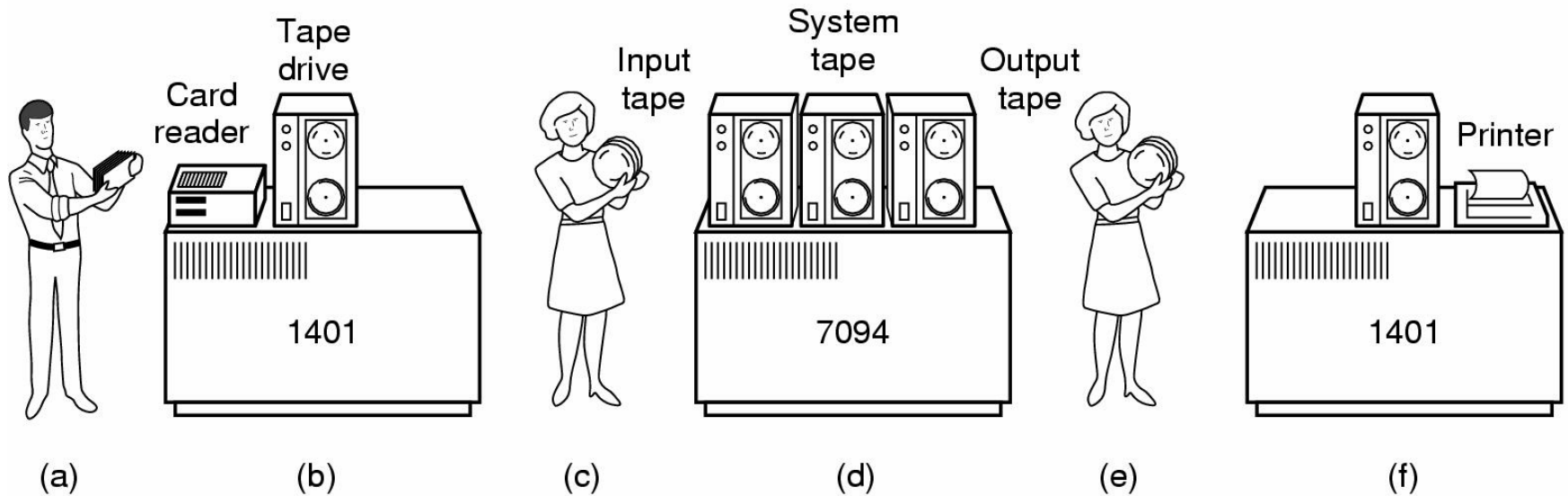
Evolução dos Sistemas Operacionais

■ Anos 50

- O transistor possibilita a criação de computadores mais confiáveis
- Programador não precisa mais operar a máquina
- Introdução do conceito de processamento de lote (**batch**) com o programa **monitor**
- Processamento de **lote** reduz o tempo entre **jobs** seqüenciais

Evolução dos Sistemas Operacionais

■ Anos 50 → Sistema Batch



Evolução dos Sistemas Operacionais

■ Anos 60

- Multiprogramação para reduzir o tempo em que o processador fica ocioso
- Memória é dividida em partições para que cada **job** fique em uma partição
- O trabalho de manuseio com fitas magnéticas é reduzido drasticamente (***spooling***)

Partições da memória

Job 1
Job 2
Job 3
Sistema Operacional

Evolução dos Sistemas Operacionais

■ Anos 70

- *Timesharing* → usuários compartilham um processador de forma transparente por meio de terminais
- Interactividade é conseguida de forma eficiente

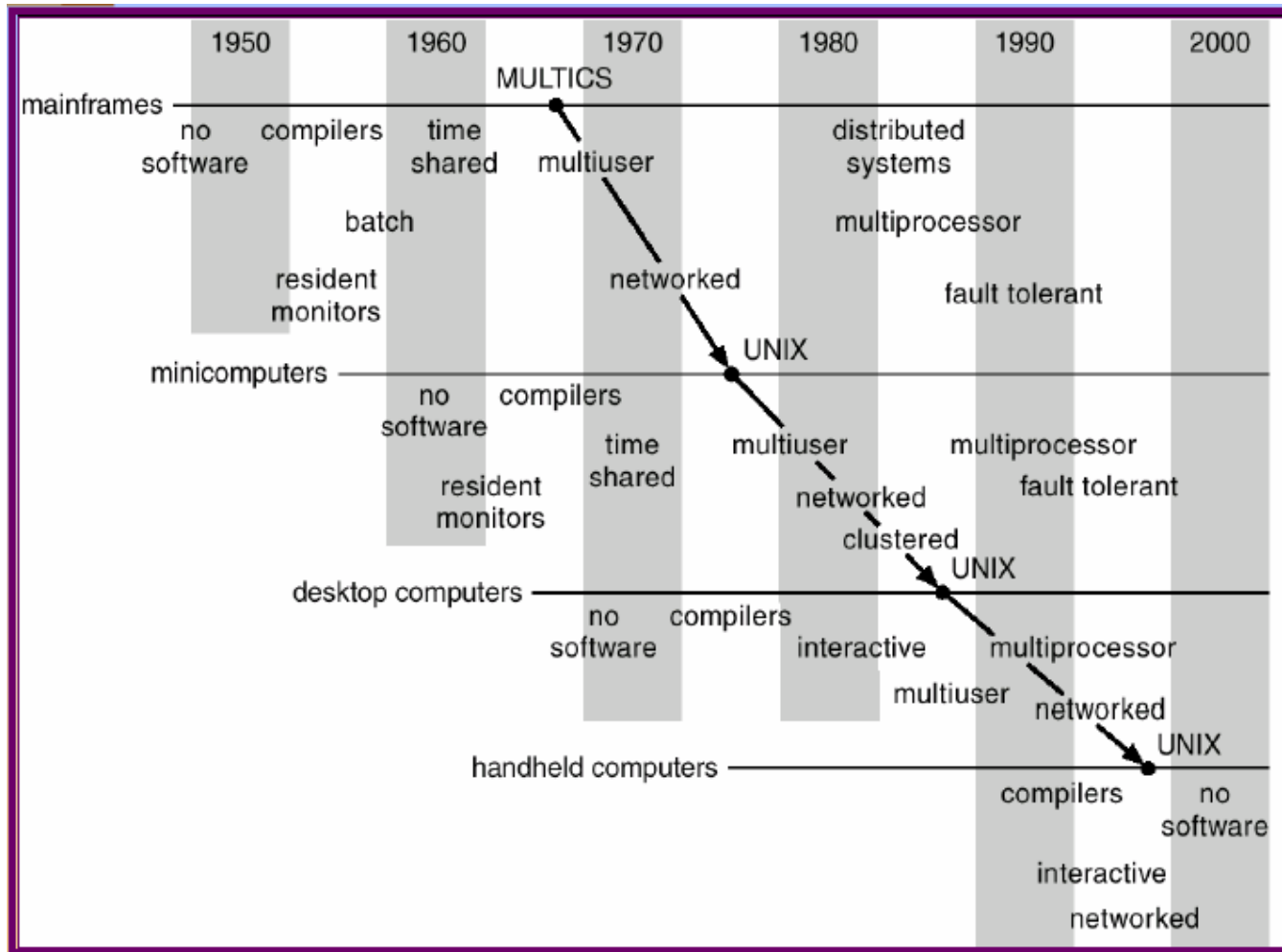
■ Anos 80

- Além de eficiência, conveniência para o usuário é procurada

■ Desde os anos 90

- Computadores PC tornam-se populares
- Sistemas Operacionais que controlam banco de dados
- Sistemas distribuídos
- Sistemas Operacionais de tempo real
- Paralelismo

Evolução dos Sistemas Operacionais (cont.)

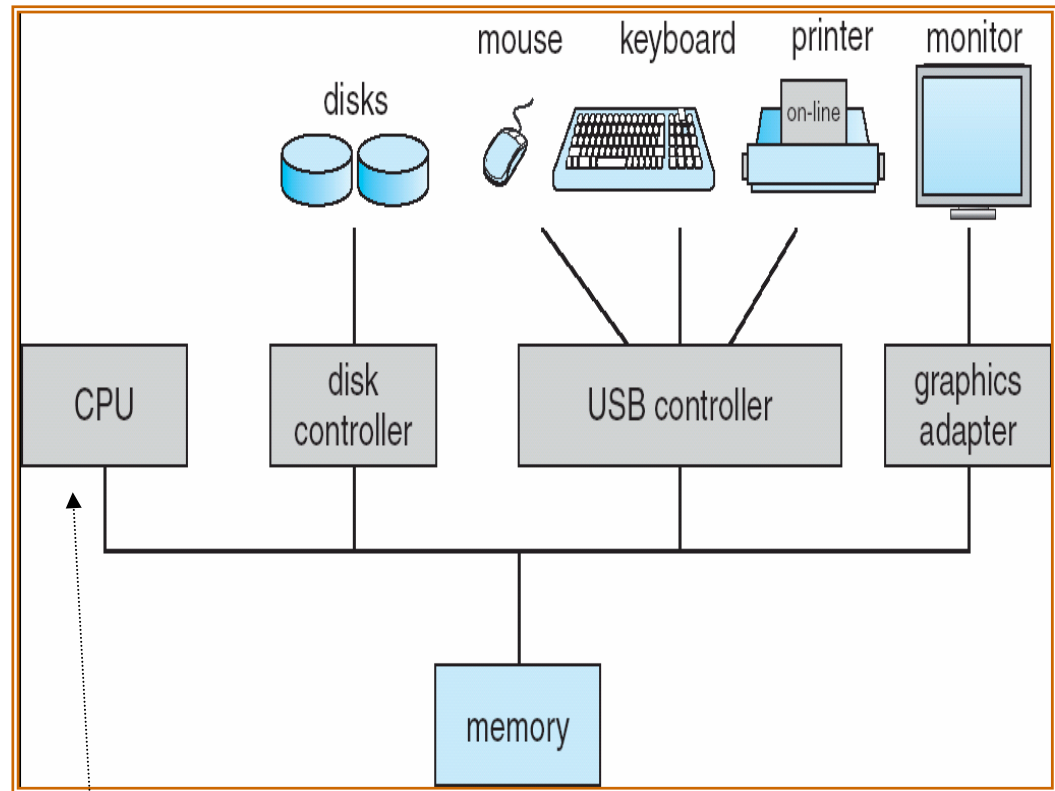


Tipos de Sistemas Operacionais

- **Main frames**
 - Jobs em batch, transações, multiprogramação
- **Servidores**
 - Usuários conectados remotamente por redes
- **Ambientes multiprocessados**
 - Extensão de SO em servidores (programação paralela)
- **Computadores pessoais**
 - Prioriza interface com o usuário
- **Aplicações em tempo real**
 - Confiabilidade de resposta previsível é priorizada
- **Sistemas embutidos**
 - Limitados em memória, potência. Ex.: palmtop, celular, etc.
- **Smart cards**
 - Muito limitados, mas alguns podem rodar até JAVA !

Revisão de Hardware

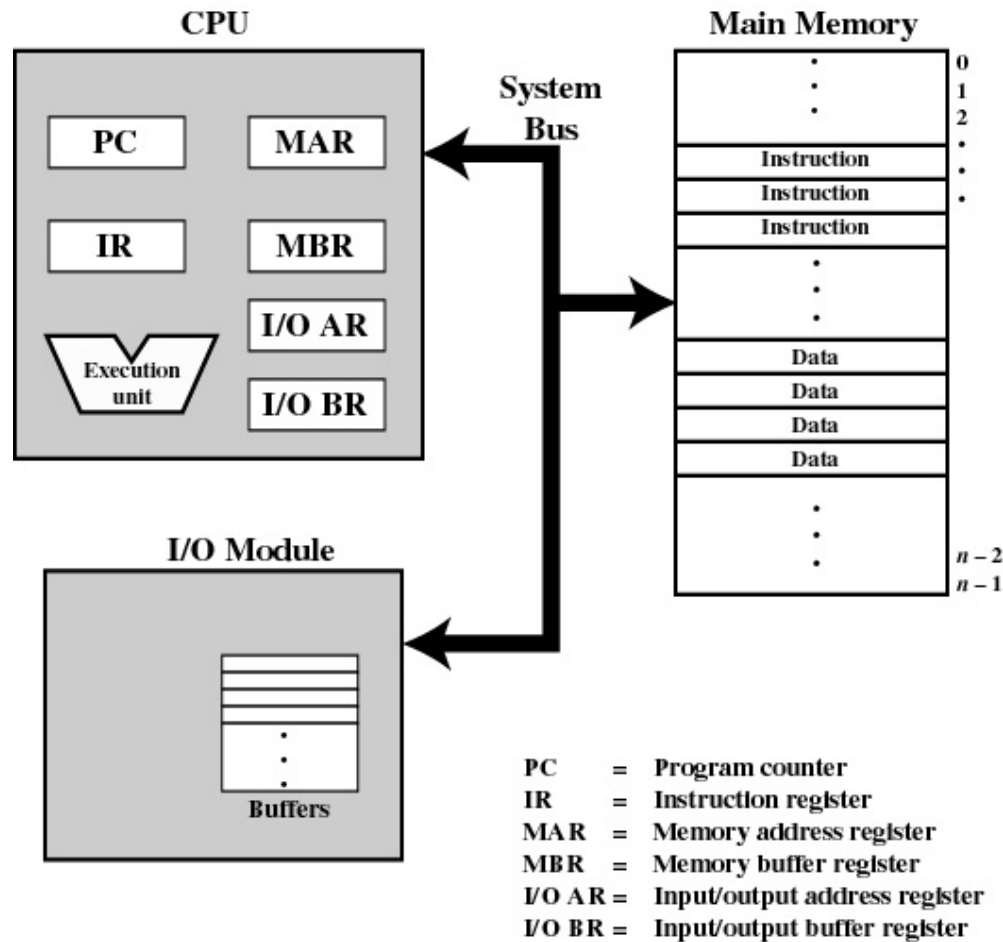
- **Todo hardware é composto de:**
 - Processador (CPU)
 - Memória
 - Periféricos (I/O)
 - Barramento (bus)
- Cada periférico tem seu controlador
- A comunicação entre todos os elementos se dá pelo barramento



Coração do computador !

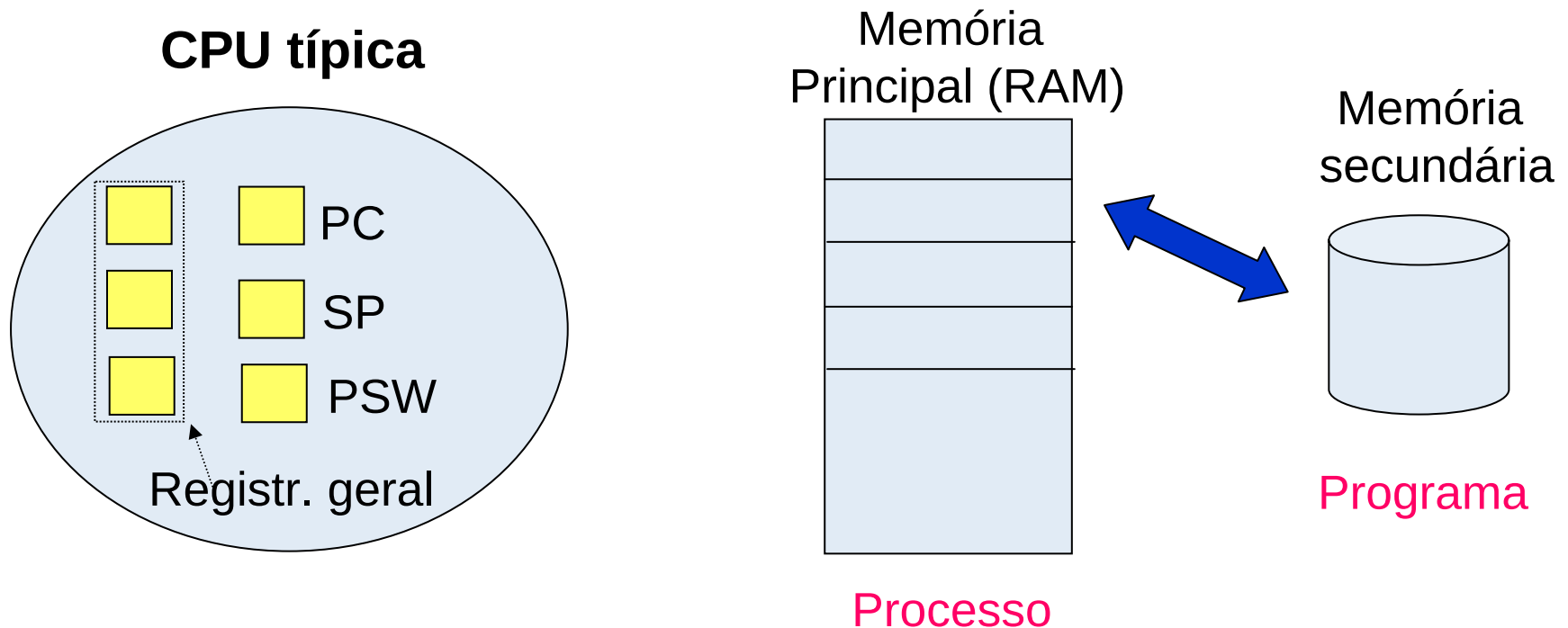
Revisão de Hardware (cont.)

- elementos básicos de um sistema de computador



Revisão de Hardware: Processos

- Processos são programas em execução
- Cada CPU tem seu conjunto de instruções próprias
- CPU possui: registradores de uso geral, registrador PC, registrador de pilha (SP), e registrador PSW



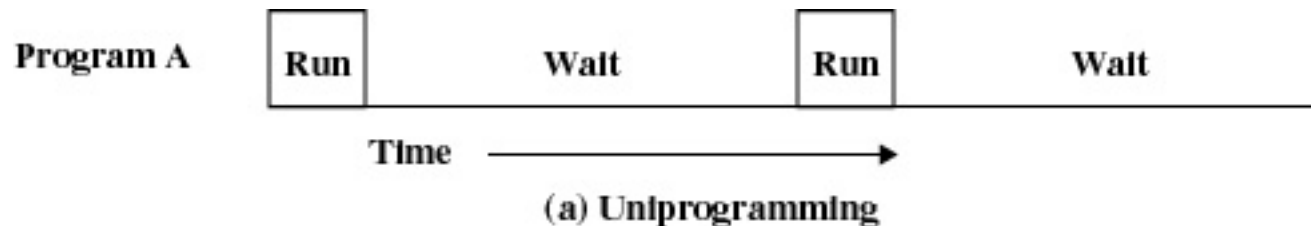
Revisão de Hardware: Processos (cont.)

- Processos podem rodar em dois modos:
 - Kernel
 - Usuário
- Um bit no registrador PSW controla o modo do processo
- Para obter acesso aos periféricos o processo precisa chavear para modo kernel
- Esse chaveamento é feito pelas chamadas de sistema (*systems calls*)
- Existem várias *systems calls* oferecidas pelo SO
- Depois de executado uma *systems calls*, o processo retorna ao modo usuário

Revisão de Hardware: Processos (cont.)

Systema monoprogramável

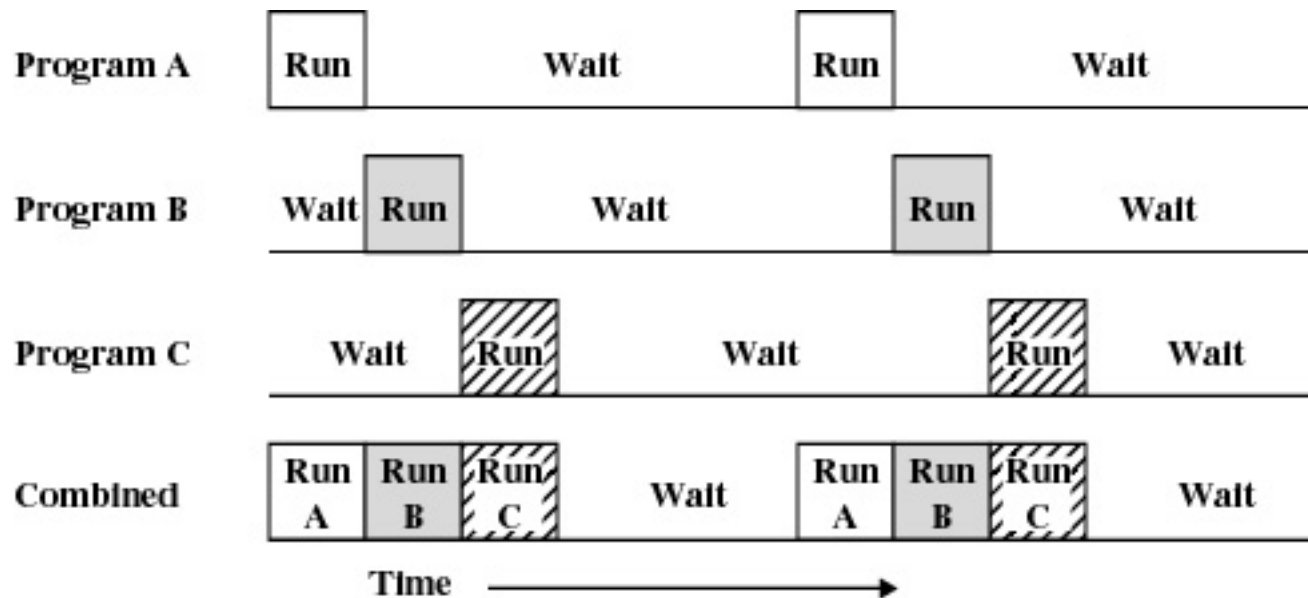
- Processo precisa esperar pelo fim da operação de I/O para prosseguir !!!



Revisão de Hardware: Processos (cont.)

Systema multiprogramável

- Quando um processo precisa esperar pelo fim da operação de I/O, o processador pode rodar outro processo !!!



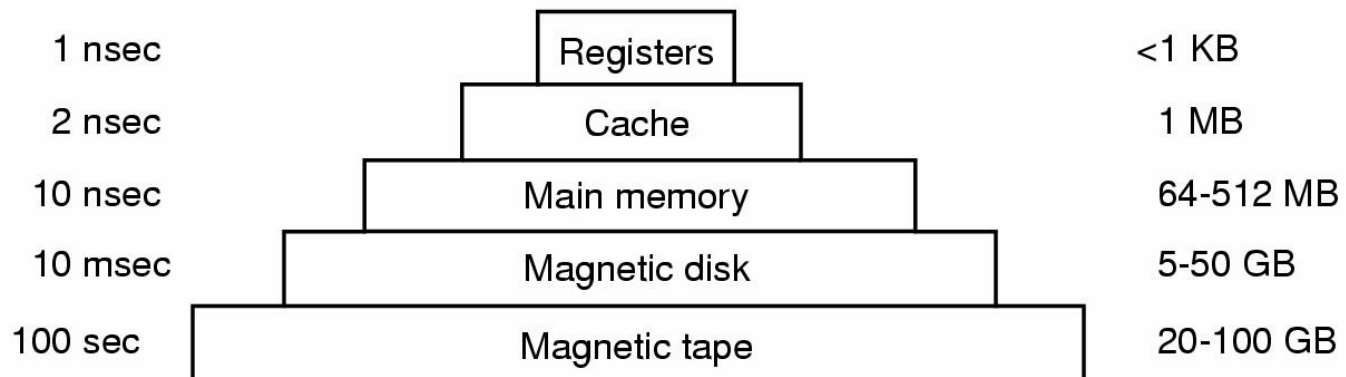
(c) Multiprogramming with three programs

Revisão de Hardware: memória

- As memórias atuais não satisfazem todos os requisitos ao mesmo tempo, que são:
 - Extremamente rápidas
 - Muito espaçosas
 - De baixíssimo custo
- Combinação de várias tecnologias
- Velocidades de acesso são inversamente proporcionais à capacidade de armazenamento

Typical access time

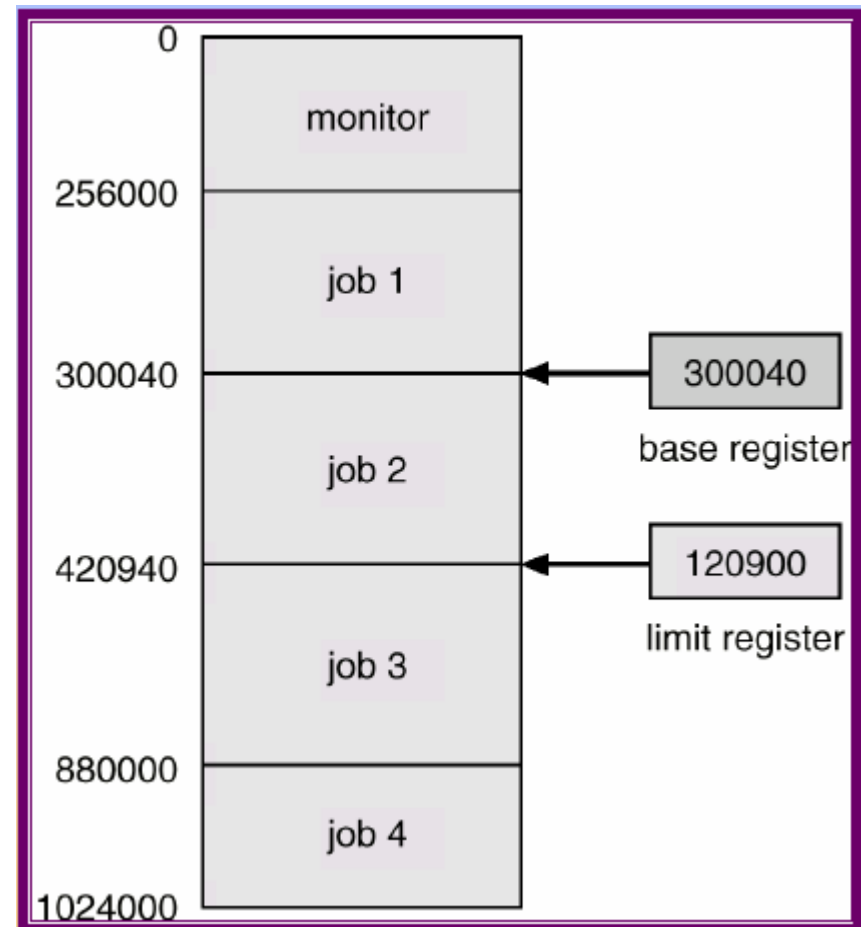
Typical capacity



Revisão de Hardware: memória (cont.)

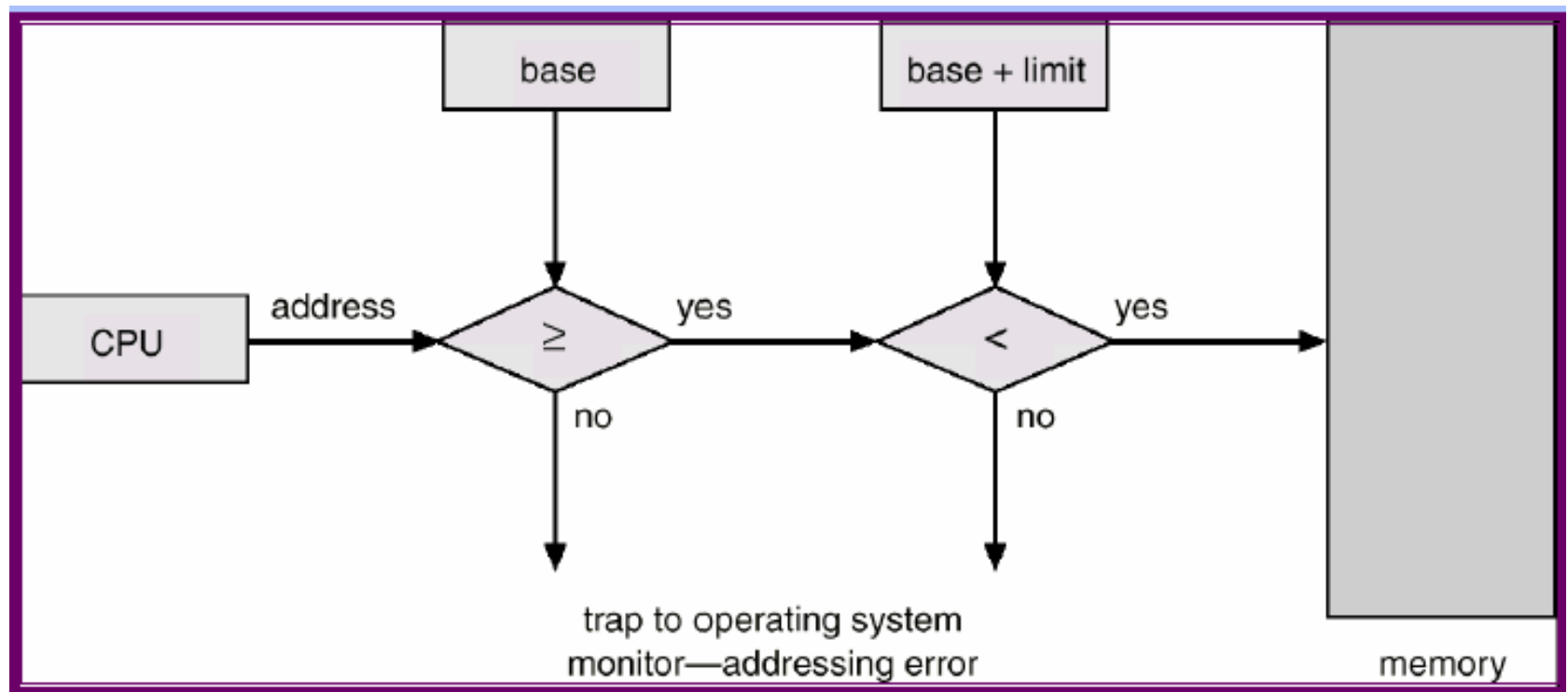
- Proteção de região de memória é fundamental em sistema multiprogramados
- Registrador base
 - Define o endereço inicial da região alocada ao proc.
- Registrador limite
 - Define o tamanho da região desde sua base
- Violação de proteção gera uma chamada ao SO

A MMU (Memory Management Unit) é quem gerência a memória



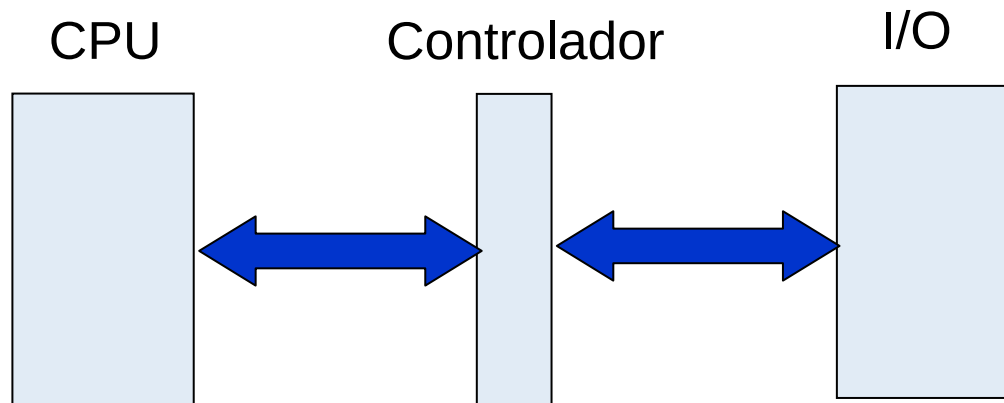
Revisão de Hardware: memória (cont.)

Se endereço vindo da CPU extrapola os limites, a MMU gera uma interrupção (trap) ao SO



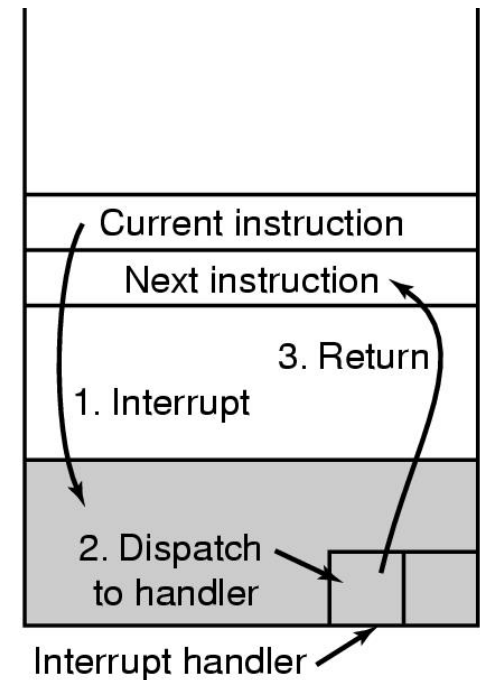
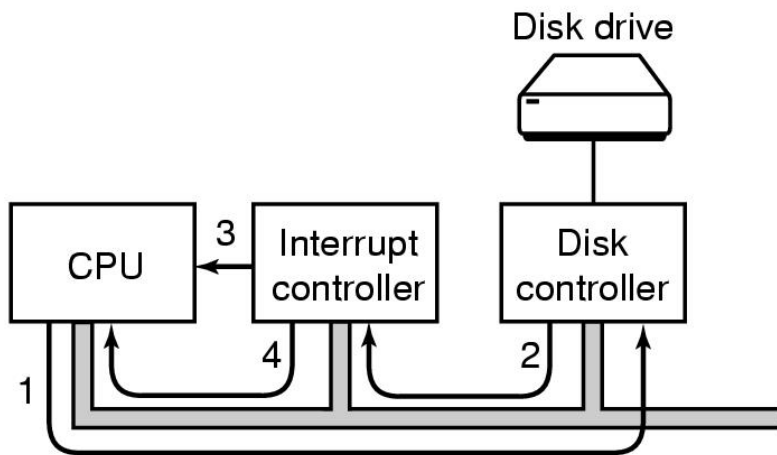
Revisão de Hardware: I/O

- Detalhes de complicações, tais como ler um arquivo em disco, ficam “escondidos” dos processos em modo usuário
- Interface padrão é usada
- Drivers são necessários adaptar mecanismos ao SO
- Drivers executam em modo kernel (são inseridos no kernel)
- Normalmente há instruções especiais IN e OUT para comunicação com os I/Os



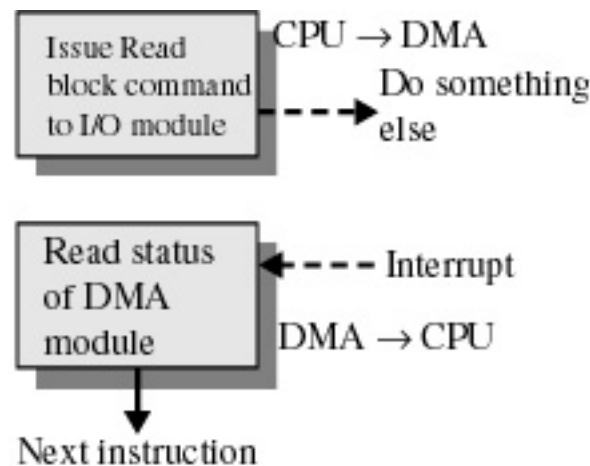
Revisão de Hardware: I/O (cont.)

- **Interrupções** são usadas para uma eficiente comunicação entre periféricos e a CPU
- CPU solicita serviço do I/O e continua com outras atividades. Uma sinalização é gerada pelo I/O depois
- Sob interrupção → PC e PSW são guardados na pilha



Revisão de Hardware: I/O (cont.)

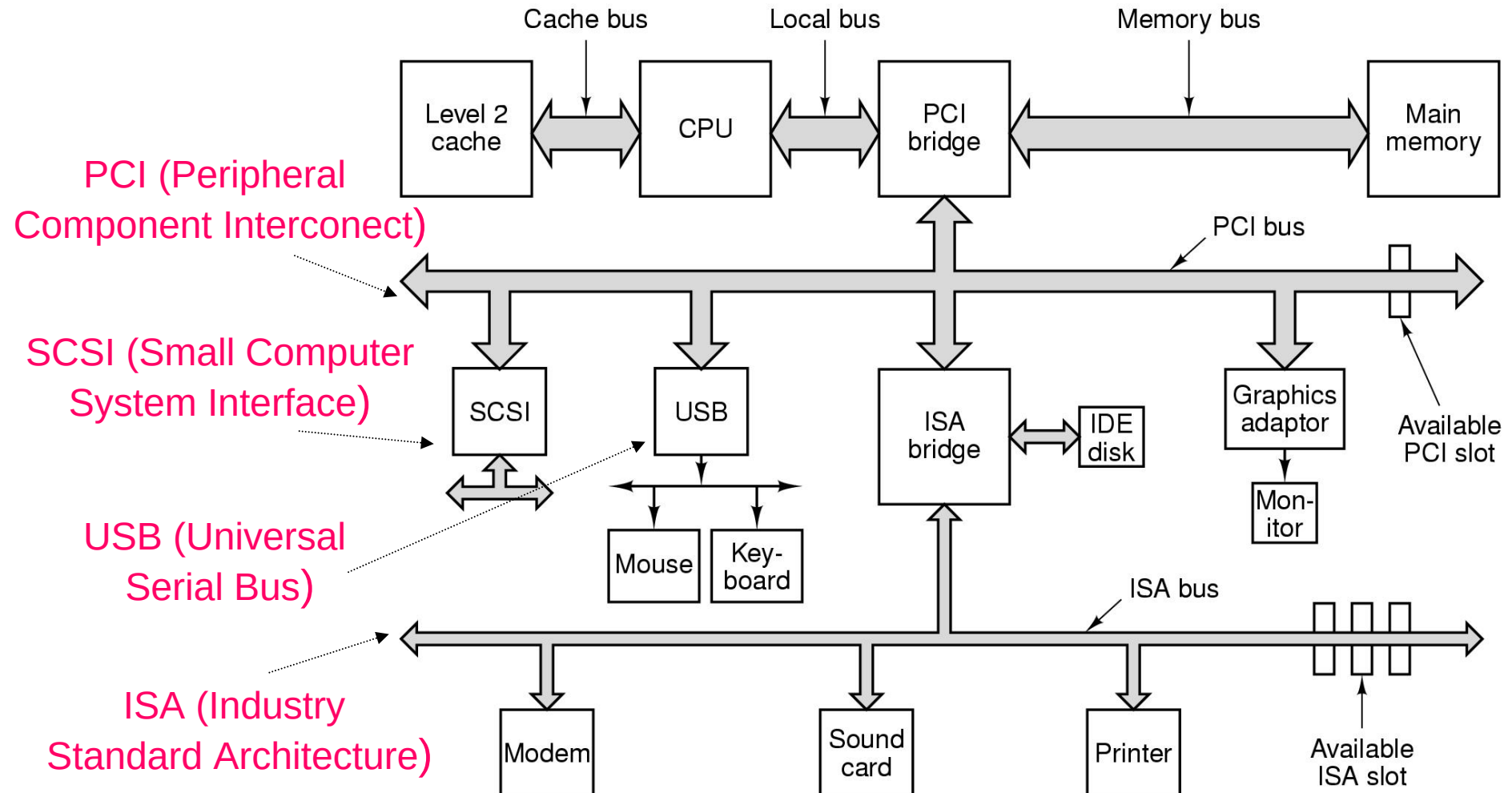
- **DMA (Direct Memory Access)** é usada para se prover acesso direto à memória sem a necessidade de intervenção da CPU (só no início)
- O controlador de disco recebe autorização da CPU para proceder com a leitura/gravação na memória diretamente
- CPU fica livre para realizar outras tarefas



(c) Direct memory access

Revisão de Hardware: Barramentos (Buses)

- Sistemas atuais possui vários barramentos para troca de informação entre os seus elementos (Ex.: **Pentium**)



System Calls (chamadas de sistema)

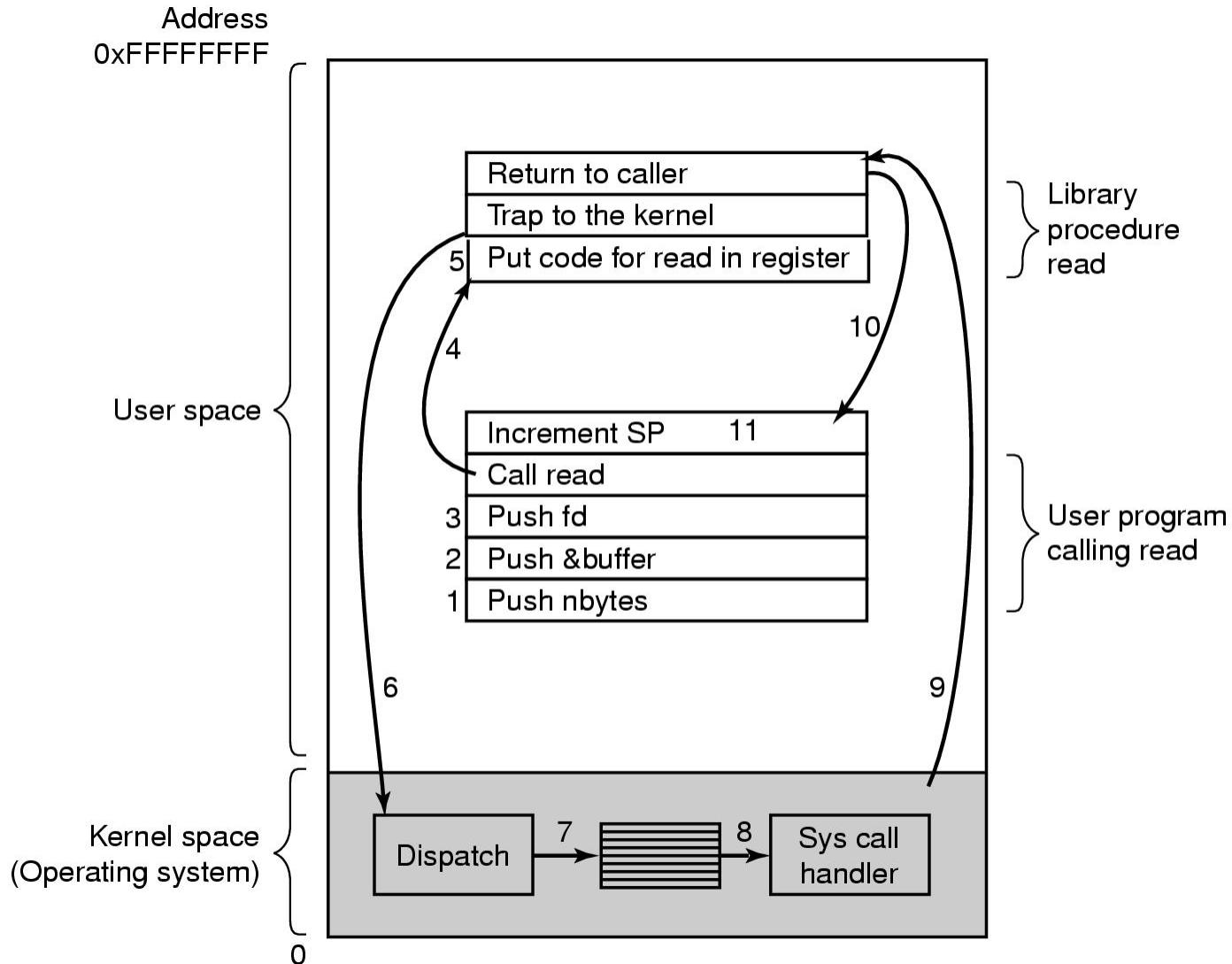
- Representam a interface entre o sistema operacional e os programas de usuários
- O SO provê várias *Systems Calls* (rodam em mod kernel) principalmente para tarefas associadas a periféricos
- Exemplo de uma *Systems Call* para leitura de arquivo
`read (fd, buffer, nbytes)`

fd=descrição do arquivo,

buffer=local para onde o dado lido deve ser armazenado,
nbytes=quantidade de bytes a serem lidos

Onze passos são necessários para a realização desta *system call*, como mostrado a seguir

System Calls (chamadas de sistema) (cont.)



System Calls para gerenciamento de processos

System Calls do UNIX

Process management

Call	Description
<code>pid = fork()</code>	Create a child process identical to the parent
<code>pid = waitpid(pid, &statloc, options)</code>	Wait for a child to terminate
<code>s = execve(name, argv, environp)</code>	Replace a process' core image
<code>exit(status)</code>	Terminate process execution and return status

System Calls para gerenciamento de arquivos

File management

Call	Description
<code>fd = open(file, how, ...)</code>	Open a file for reading, writing or both
<code>s = close(fd)</code>	Close an open file
<code>n = read(fd, buffer, nbytes)</code>	Read data from a file into a buffer
<code>n = write(fd, buffer, nbytes)</code>	Write data from a buffer into a file
<code>position = lseek(fd, offset, whence)</code>	Move the file pointer
<code>s = stat(name, &buf)</code>	Get a file's status information

System Calls para gerenciamento de diretórios

Directory and file system management

Call	Description
s = mkdir(name, mode)	Create a new directory
s = rmdir(name)	Remove an empty directory
s = link(name1, name2)	Create a new entry, name2, pointing to name1
s = unlink(name)	Remove a directory entry
s = mount(special, name, flag)	Mount a file system
s = umount(special)	Unmount a file system

Outras System Calls

Miscellaneous

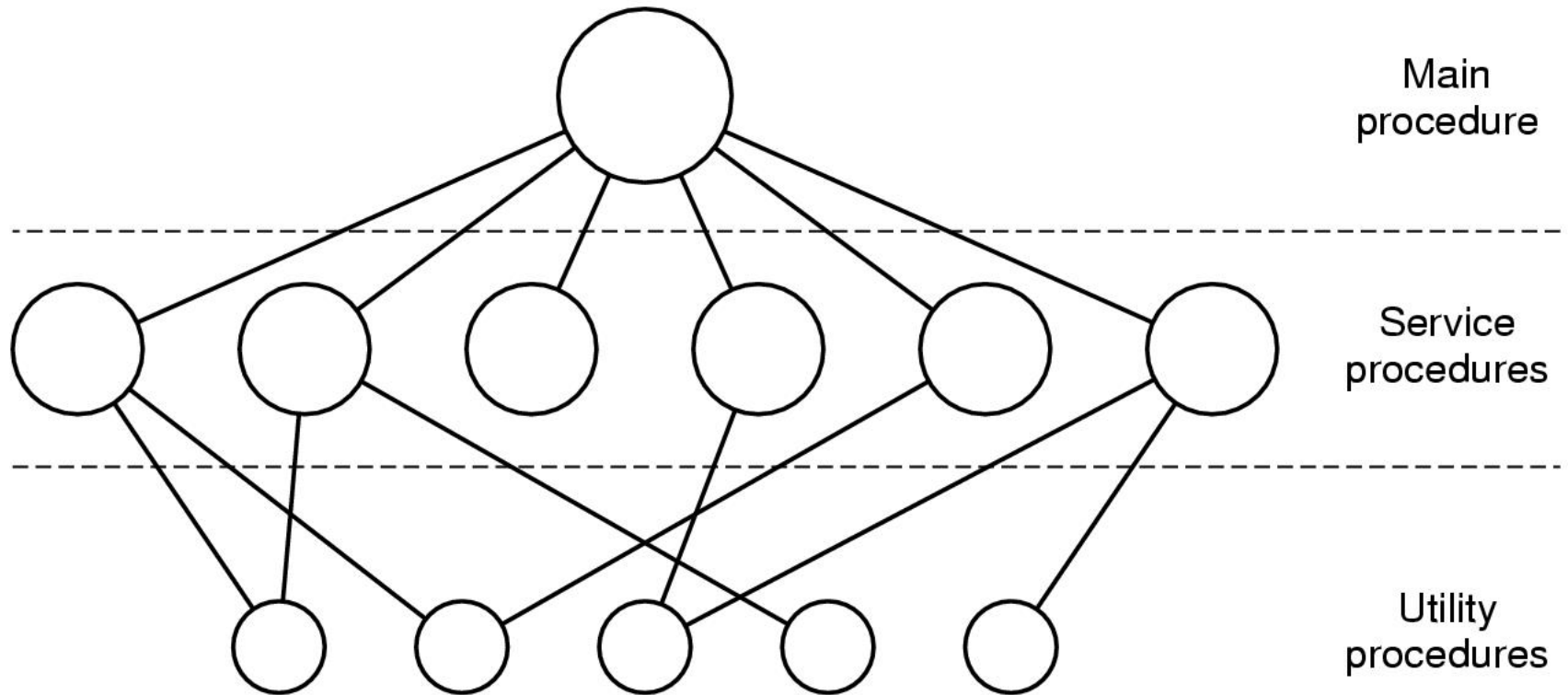
Call	Description
<code>s = chdir(dirname)</code>	Change the working directory
<code>s = chmod(name, mode)</code>	Change a file's protection bits
<code>s = kill(pid, signal)</code>	Send a signal to a process
<code>seconds = time(&seconds)</code>	Get the elapsed time since Jan. 1, 1970

System Calls do Windows

UNIX	Win32	Description
fork	CreateProcess	Create a new process
waitpid	WaitForSingleObject	Can wait for a process to exit
execve	(none)	CreateProcess = fork + execve
exit	ExitProcess	Terminate execution
open	CreateFile	Create a file or open an existing file
close	CloseHandle	Close a file
read	ReadFile	Read data from a file
write	WriteFile	Write data to a file
lseek	SetFilePointer	Move the file pointer
stat	GetFileAttributesEx	Get various file attributes
mkdir	CreateDirectory	Create a new directory
rmdir	RemoveDirectory	Remove an empty directory
link	(none)	Win32 does not support links
unlink	DeleteFile	Destroy an existing file
mount	(none)	Win32 does not support mount
umount	(none)	Win32 does not support mount
chdir	SetCurrentDirectory	Change the current working directory
chmod	(none)	Win32 does not support security (although NT does)
kill	(none)	Win32 does not support signals
time	GetLocalTime	Get the current time

Estruturas do Sistema Operacional

- **Monolítico:** desestruturado porque os processos se comunicam diretamente entre si



Estruturas do Sistema Operacional (cont.)

- **Em camadas:**

- cada camada desenvolve uma atividade específica
- Há uma interface padrão para comunicação entre as camadas adjacentes

Layer	Function
5	The operator
4	User programs
3	Input/output management
2	Operator-process communication
1	Memory and drum management
0	Processor allocation and multiprogramming

Leitura Complementar

- ***Sistemas Operacionais Modernos, 2ª edição***
Andrew Tanenbaum, Editora Pearson, 2003.
- **Operating Systems (4th edition),**
William Stallings, Prentice Hall, 2001
- ***Sistemas Operacionais - Conceitos e Aplicações,***
A. Silberschatz, P. Galvin, e G. Gane, Editora Campus, 2001.
- **Sistemas Operacionais**
R. S. Oliveira, A. Carissimi e S. Toscani, Editora Sagra Luzzatto, 2004